

1 Introdução

A Busca em Largura, ou *Breadth-First Search* (BFS), é um dos algoritmos fundamentais de travessia em grafos [5]. O algoritmo parte de um vértice inicial e, em seguida, visita os demais vértices em camadas: primeiro os que estão a uma aresta de distância, depois os que estão a duas arestas, e assim sucessivamente. Dessa forma, em um grafo não ponderado, a BFS calcula a menor distância, em número de arestas, entre o vértice inicial e todos os vértices alcançáveis a partir dele. A Busca em Largura também gera uma árvore chamada *Árvore de Busca em Largura*, na qual o pai de um vértice é aquele que o descobriu durante a busca.

A implementação clássica utiliza uma fila. O vértice inicial é inserido na fila, marcado como visitado, e sua distância é definida como zero. Em seguida, enquanto a fila não estiver vazia, repete-se a seguinte operação retira-se um vértice v da fila e examinam-se todos os vértices u vizinhos de v . Caso u ainda não tenha sido visitado, sua distância é calculada, ele é marcado como visitado e adicionado à fila.

Uma forma simplificada do algoritmo é apresentada a seguir:

```
BFS(G, origem):
    para cada vertice v em G:
        pai[v] = none

    pai[origem] = origem
    fila = {origem}

    enquanto fila nao esta vazia:
        u = proximo elemento da fila
        para cada vizinho v de u:
            se pai[v] == none:
                pai[v] = u
                adiciona v na fila
```

2 Problemas Reais

Grafos aparecem naturalmente em problemas reais de grande escala. Em uma rede social, vertices podem representar usuarios e arestas podem representar amizades, seguidores, mensagens ou interacoes [6]. Na Web, paginas sao vertices e hiperlinks sao arestas [4]. Em seguranca computacional, grafos podem modelar fluxos de rede, dependencias entre processos, tentativas de acesso e relacoes entre entidades suspeitas [1]. Em biologia, vertices podem representar proteinas, genes ou moléculas, e arestas podem representar interacoes [2]. Em todos esses casos, a BFS aparece como uma primitiva para calcular alcancabilidade, distancia em numero de arestas e expansao de vizinhanca.

A literatura de sistemas de grafos trata a BFS como uma das operacoes centrais para avaliar plataformas de processamento de dados irregulares. O Graph500 foi proposto para complementar benchmarks numericos tradicionais com uma carga de trabalho orientada a dados, motivado pela diferenca entre

simulacoes fisicas, que tendem a ser estruturadas e intensivas em ponto flutuante, e aplicacoes de informatica, que tendem a ser nao estruturadas, orientadas a inteiros e com baixa localidade [12]. Sua especificacao define a BFS como o Kernel 2 do benchmark, executado sobre grafos sinteticos de escala crescente, e usa a metrica TEPS, ou arestas atravessadas por segundo, para medir desempenho [7]. Isso mostra que a BFS nao e apenas um exercicio teorico: ela e usada como proxy para medir a capacidade de supercomputadores processarem analise de grafos massivos.

Em redes sociais, a BFS modela consultas de vizinhanca e distancia social. A pergunta “quais usuarios estao a ate dois ou tres saltos de uma conta?” e uma execucao limitada de BFS. Esse tipo de consulta aparece em recomendacao de amizades, propagacao de informacao, deteccao de comunidades locais e analise de influencia [6]. Em grafos com bilhoes de relacoes, a expansao de poucos niveis pode atingir uma quantidade enorme de vertices, especialmente quando existem hubs de grau muito alto. Por isso, mesmo uma BFS limitada por profundidade pode exigir processamento paralelo e estruturas de dados distribuidas.

Em grafos da Web, a BFS tambem e uma abstracao natural. Um rastreador *web crawler* pode iniciar em um conjunto de paginas sementes e expandir links por camadas, descobrindo novas paginas a cada nivel [4]. O sistema Pregel, do Google, foi proposto para processamento de grafos em larga escala e apresenta algoritmos como caminhos minimos, PageRank, propagacao de rotulos e outras computacoes iterativas sobre grafos distribuidos [9]. Embora Pregel seja um modelo mais geral que BFS, sua execucao em superpassos e troca de mensagens entre vertices e muito proxima da ideia de fronteiras sucessivas usada em BFS distribuida.

Em seguranca, investigacao de fraudes e analise de dependencias, BFS e usada para expandir a vizinhanca de uma entidade de interesse. A partir de uma conta, um endereco IP, uma maquina infectada ou um processo suspeito, a busca em largura identifica entidades conectadas por poucos saltos. Essa forma de exploracao e importante porque muitas relacoes relevantes nao aparecem em uma aresta direta, mas em cadeias curtas: contas que compartilham dispositivos, maquinas que se comunicam com o mesmo servidor, ou processos que dependem dos mesmos arquivos [1]. Em grafos corporativos ou de telecomunicacoes, essas buscas podem envolver centenas de milhoes de vertices e precisam responder em tempo interativo ou quase interativo.

O desafio e que os grafos modernos podem ser massivos e irregulares. Grafos de redes sociais e da Web frequentemente possuem distribuicao de grau altamente desbalanceada: poucos vertices concentram muitas arestas, enquanto a maioria tem grau baixo. Essa propriedade gera duas dificuldades para BFS. Primeiro, ha desbalanceamento de trabalho, pois expandir um vertice de alto grau custa muito mais do que expandir um vertice comum. Segundo, ha muita contencao, pois muitos caminhos podem tentar descobrir o mesmo vertice ao mesmo tempo.

Alem disso, a BFS tem baixa localidade de memoria. Em matrizes densas ou vetores, os acessos costumam ser contiguos e previsiveis. Em grafos, ao expandir a fronteira, o algoritmo segue listas de adjacencia que podem apontar

para posicoes distantes e aparentemente aleatorias. Consequentemente, a BFS e normalmente limitada por largura de banda, latencia de memoria, atomicos, sincronizacao e comunicacao entre maquinas, e nao por operacoes aritmeticas.

Trabalhos em arquiteturas paralelas mostram como essas dificuldades aparecem na pratica. Beamer, Asanovic e Patterson propuseram a BFS *direction optimizing*, que alterna entre uma expansao top-down, baseada na fronteira atual, e uma expansao bottom-up, que examina vertices ainda nao visitados procurando vizinhos na fronteira [3]. Essa mudanca reduz trabalho em grafos de pequeno diametro e grande fronteira, um caso comum em grafos sociais e da Web. Merrill, Garland e Grimshaw estudaram BFS em GPU e mostraram que a primitiva e representativa de computacoes paralelas com acesso a memoria e distribuicao de trabalho irregulares; sua implementacao alcançou bilhoes de arestas atravessadas por segundo em uma ou mais GPUs [10].

Portanto, BFS aparece em problemas reais de duas formas complementares. Como algoritmo final, ela responde consultas de alcance, distancia e vizinhanca. Como primitiva de sistema, ela estressa exatamente os componentes que tornam grafos massivos dificeis: memoria irregular, fronteiras dinamicas, atomicos, desbalanceamento e comunicacao. Por isso, estudar BFS em GPU e em ambientes distribuidos e uma forma direta de estudar o desempenho de analise de grafos em escala real.

3 BFS em Fronteiras

O algoritmo de BFS também pode ser implementado como uma descoberta por fronteiras. Inicializa-se a primeira fronteira com o vértice inicial. Em seguida, para cada vértice presente na fronteira atual, examinam-se todos os seus vizinhos; aqueles que ainda não foram visitados são adicionados à próxima fronteira. Esse processo se repete sucessivamente, até que a fronteira atual esteja vazia.

Uma versão simplificada do algoritmo é apresentada a seguir:

```
BFS(G, origem):
    para cada vertice v em G:
        pai[v] = none

    pai[origem] = origem
    fronteira = {origem}

    enquanto fronteira nao esta vazia:
        proxima_fronteira = vazia
        para cada vertice u na fronteira:
            para cada vizinho v de u:
                se pai[v] == none:
                    pai[v] = u
                    adiciona v na proxima_fronteira
        fronteira = proxima_fronteira
```

3.1 Exemplo Visual de Fronteiras

Considere o grafo da Figura 1. A busca começa no vertice A. Em cada passo, os vertices em azul representam a fronteira atual, os vertices em cinza já foram visitados em níveis anteriores, e os vertices em branco ainda não foram descobertos. As arestas destacadas em azul formam a árvore de BFS construída até aquele momento. Essa árvore registra, para cada vertice descoberto, qual foi o vertice responsável por sua primeira descoberta. Assim, cada aresta azul liga um vertice ao seu pai na busca, permitindo recuperar um caminho mínimo, em número de arestas, da origem A até qualquer vertice alcançável.

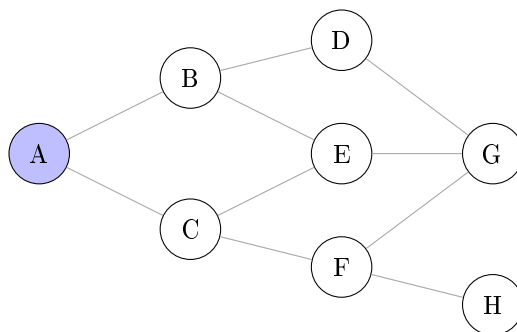


Figure 1: Passo 0 da BFS: a fronteira inicial contém apenas a origem A.

No passo inicial, a distância de A é zero. A primeira expansão examina as arestas que saem de A. Como B e C ainda não foram visitados, ambos são descobertos, recebem distância um e passam a formar a próxima fronteira.

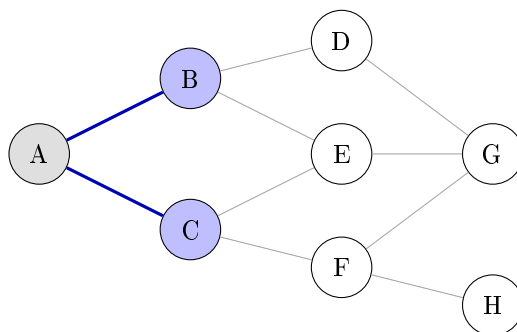


Figure 2: Passo 1 da BFS: B e C formam a fronteira de nível 1.

No segundo passo, a BFS expande todos os vértices da fronteira atual. O vértice B descobre D e E. O vértice C também possui uma aresta para E, mas E já pode ter sido descoberto por B; nesse caso, a implementação mantém apenas um pai para E. O vértice C também descobre F. Assim, a nova fronteira passa a ser formada por D, E e F, todos com distância dois.

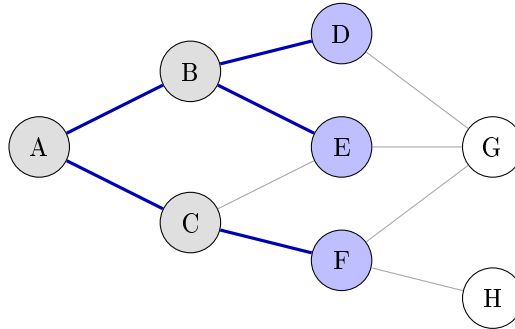


Figure 3: Passo 2 da BFS: D, E e F formam a fronteira de nível 2.

No terceiro passo, a fronteira de nível dois é expandida. Os vertices D, E e F possuem arestas para G. O primeiro que conseguir registrar G como descoberto define seu pai na árvore de BFS. Na figura, E foi escolhido como pai de G, mas D ou F também poderiam ser pais válidos, pois todos pertencem ao nível anterior. Além disso, F descobre H, que também passa a fazer parte da fronteira de nível três.

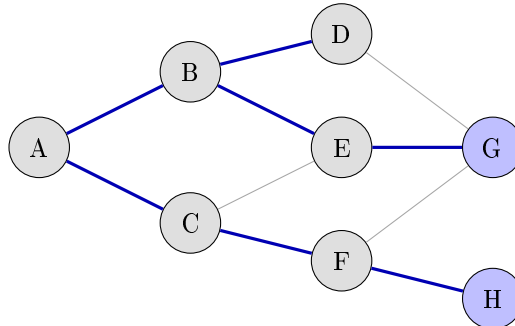


Figure 4: Passo 3 da BFS: G e H são descobertos no nível 3.

Depois da expansão de G, nenhum novo vertice é encontrado. A próxima fronteira fica vazia e o algoritmo termina. O resultado é uma árvore de BFS na qual cada vertice alcançável possui um pai em um nível imediatamente anterior ao seu.

4 Motivação de Paralelismo

A BFS baseada em fronteiras oferece uma oportunidade natural de paralelismo, todos os vertices de uma mesma fronteira podem ser processados simultaneamente, e suas arestas podem ser expandidas em paralelo. O principal cuidado é controlar a descoberta dos vertices da próxima fronteira, pois múltiplas arestas

podem tentar descobrir o mesmo vertice ao mesmo tempo. Estrategias de paralelismo e controle foram discutidas em [10].

4.1 Divisão do Grafo entre Workers

Neste texto, o termo *worker* e usado de forma geral para representar uma unidade de execucao capaz de armazenar parte do grafo e realizar parte do trabalho da BFS. Um worker pode corresponder, por exemplo, a um processo, uma maquina, uma GPU, uma CPU ou outro recurso de execucao paralelo. Os detalhes de como esses workers sao implementados serao discutidos posteriormente. Essa abstracao e comum em sistemas de processamento paralelo de grafos, nos quais a computacao e dividida entre unidades que armazenam partes do grafo e trocam informacoes sobre vertices descobertos durante a execucao [10].

Em uma implementação paralela ou distribuída, o grafo não precisa ficar inteiro em uma única unidade de execução. Uma estratégia comum é dividir seus vértices entre vários *workers*. Cada worker fica responsável por armazenar e processar uma parte dos vértices e de suas listas de adjacência. Durante a BFS, quando um worker expande um vértice local, ele examina suas arestas; se o vizinho pertence a outro worker, a descoberta desse vizinho precisa ser comunicada ao worker que o possui [10].

Uma forma simples de particionamento é a distribuição cíclica. Com n_w workers, o dono de um vértice pode ser calculado por:

$$\text{worker}(v) = v \bmod n_workers$$

Assim, os vértices são distribuídos de maneira alternada entre os workers. Essa estratégia não garante balanceamento perfeito para todos os grafos, mas é simples e evita concentrar intervalos inteiros de vértices em um único worker. A Tabela 1 mostra um exemplo com 16 vértices.

Worker	Vértices pertencentes ao worker
0	0, 4, 8, 12
1	1, 5, 9, 13
2	2, 6, 10, 14
3	3, 7, 11, 15

Table 1: Particionamento cíclico de 16 vértices entre 4 workers.

A Figura 5 mostra a visão global de um grafo com 16 vértices. Essa visão representa o grafo completo antes de considerar qual worker é responsável por cada vértice.

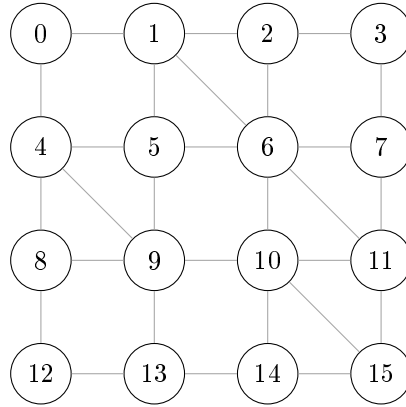


Figure 5: Visão global de um grafo com 16 vértices antes do particionamento.

A Figura 6 mostra o mesmo grafo particionado entre quatro workers. As cores indicam o dono de cada vértice. As arestas que ligam vértices de cores diferentes representam comunicações potenciais durante a BFS, pois a expansão de um vértice em um worker pode descobrir um vértice que pertence a outro worker.

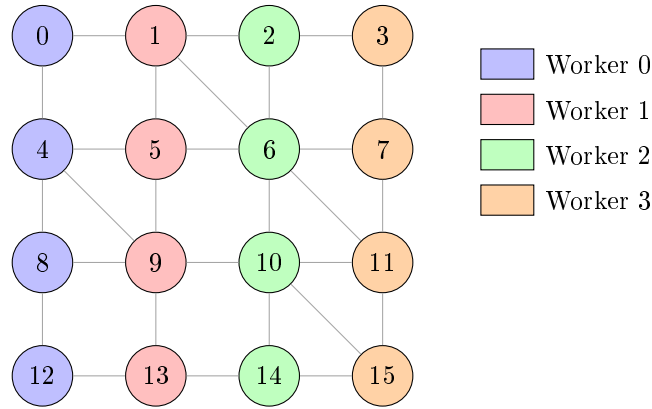


Figure 6: Mesmo grafo particionado entre 4 workers usando a regra $worker(v) = v \bmod 4$.

4.2 Exemplo de BFS Paralela com 4 Workers

Considere uma BFS iniciada no vértice 0 do grafo particionado da Figura 6. No passo inicial, apenas o worker 0 possui trabalho, pois ele é o dono do vértice 0. Ao expandir 0, são descobertos os vértices 1 e 4. O vértice 4 pertence ao próprio worker 0, enquanto o vértice 1 pertence ao worker 1. Portanto, a nova fronteira já fica distribuída entre mais de um worker.

Após a expansão dos vértices 1 e 4, a BFS chega ao passo mostrado na Figura 9. Nesse momento, a fronteira global de nível 2 é formada por vértices armazenados em workers diferentes. O worker 0 possui o vértice 8 na fronteira, o worker 1 possui os vértices 5 e 9, o worker 2 possui os vértices 2 e 6, e o worker 3 ainda não possui vértices na fronteira atual.

A expansão passo a passo é mostrada nas figuras seguintes. Em cada uma delas, a fronteira global é dividida entre os quatro workers. Os vértices em azul formam a fronteira local do passo atual, os vértices em cinza já foram visitados, e os vértices em branco ainda não foram descobertos. Depois que todos os workers expandem suas fronteiras locais, os vértices recém-descobertos formam a fronteira do próximo nível.

No passo 0, apenas o worker 0 possui trabalho, pois a origem 0 pertence a ele.

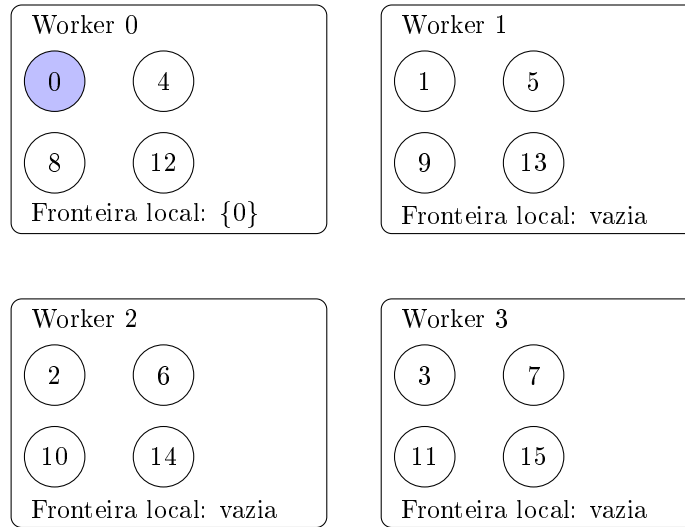


Figure 7: Passo 0 da BFS paralela: apenas a origem 0 está na fronteira.

Ao expandir 0, são descobertos 1 e 4. O vértice 4 permanece no worker 0, e o vértice 1 passa a compor a fronteira local do worker 1.

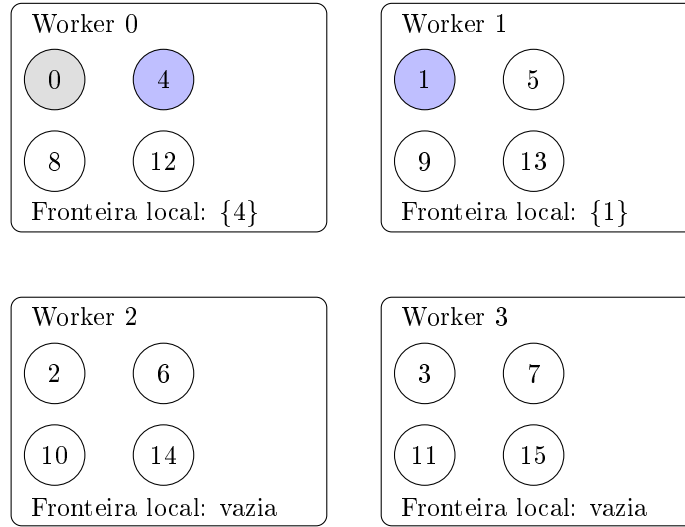


Figure 8: Passo 1 da BFS paralela: os workers 0 e 1 possuem vértices na fronteira.

No passo 1, os workers 0 e 1 trabalham ao mesmo tempo: 4 descobre 8 e 9, enquanto 1 descobre 2, 5 e 6. O vértice 5 também é vizinho de 4, mas basta que uma das tentativas de descoberta seja aceita para que ele entre na próxima fronteira. No passo 2, já há trabalho em três workers diferentes.

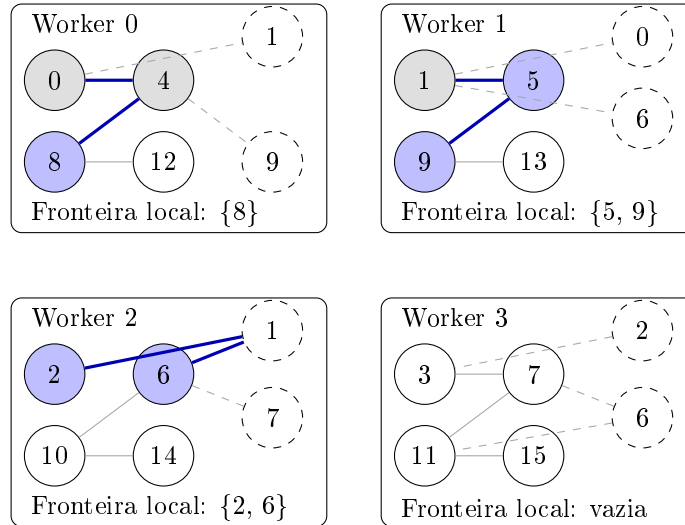


Figure 9: Passo 2 da BFS paralela: a fronteira global está distribuída entre os workers 0, 1 e 2. Vértices tracejados representam vizinhos remotos mantidos apenas para ilustrar comunicações entre workers.

No passo 2, a expansão dos vértices 2 e 6 no worker 2 gera trabalho para o worker 3, pois existem arestas para os vértices 3, 7 e 11. Ao mesmo tempo, os workers 0, 1 e 2 descobrem outros vértices locais, formando a próxima fronteira.

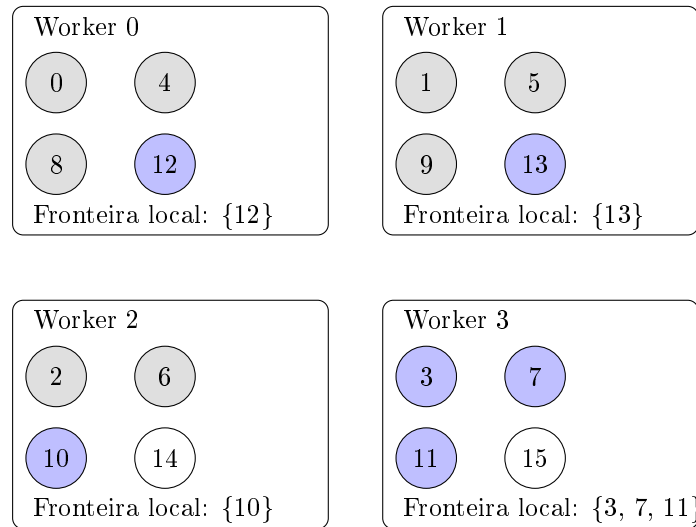


Figure 10: Passo 3 da BFS paralela: todos os workers possuem vértices na fronteira.

Após o passo 3, apenas os workers 2 e 3 continuam com novos vértices a expandir.

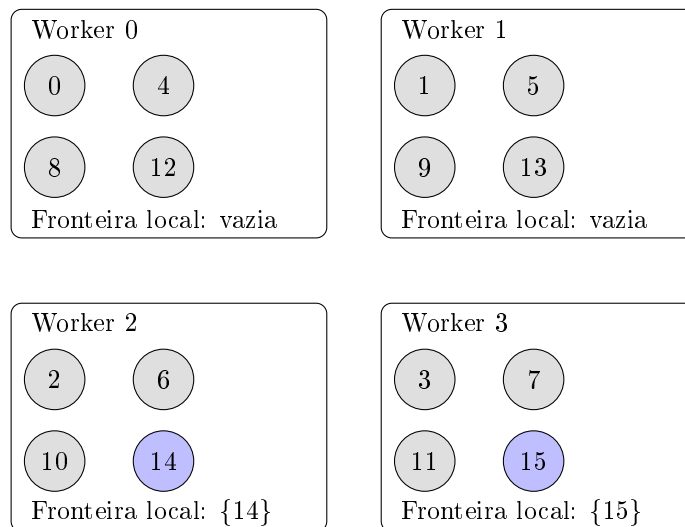


Figure 11: Passo 4 da BFS paralela: apenas os workers 2 e 3 continuam com fronteiras locais.

Por fim, a expansão de 14 e 15 não produz novas descobertas. Nesse ponto, a próxima fronteira fica vazia e a BFS termina.

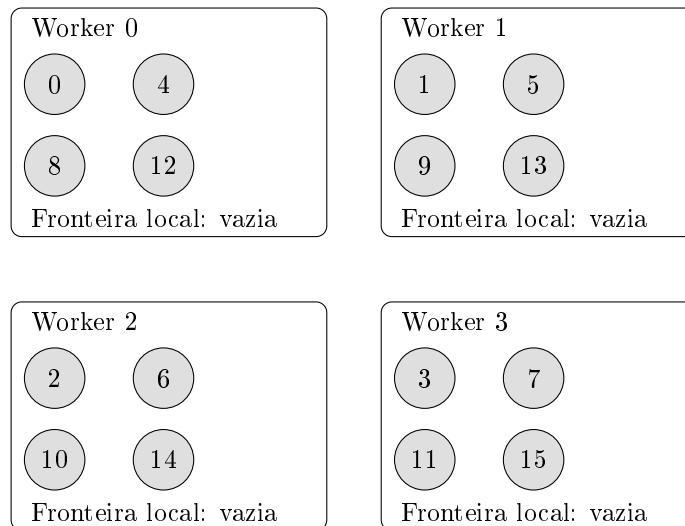


Figure 12: Passo 5 da BFS paralela: a fronteira global fica vazia e a busca termina.

Essa sequência visual mostra que a fronteira global da BFS é a união das fronteiras locais de cada worker. Em uma implementação paralela, cada worker ex-

pande apenas os vértices da sua fronteira local. Quando uma aresta aponta para um vértice remoto, a atualização precisa ser enviada ao worker dono daquele vértice, que poderá processá-lo no próximo passo. Essa organização preserva a ideia de avanço por níveis da BFS, mas distribui o trabalho e a comunicação entre as unidades de execução disponíveis.

5 Graph500

O Graph500 é um benchmark criado para avaliar sistemas de alto desempenho em cargas de trabalho centradas em grafos e acesso irregular à memória. Diferente de benchmarks voltados principalmente para operações de ponto flutuante, como os associados à álgebra linear densa, o Graph500 enfatiza movimentação de dados, comunicação, acesso indireto à memória e paralelismo irregular [12]. Essas características fazem com que ele seja adequado para estudar algoritmos como BFS em ambientes distribuídos e em GPU.

O benchmark define uma entrada sintética baseada em um grafo gerado por um modelo do tipo Kronecker/R-MAT. O tamanho do grafo é controlado pelo parâmetro **SCALE**, que define o número de vértices como 2^{SCALE} , e pelo **edgefactor**, que define a quantidade média de arestas geradas por vértice. Assim, o número de arestas cresce proporcionalmente ao número de vértices, mas a estrutura gerada não é regular: ela tende a produzir vértices com graus muito diferentes, criando hubs e regiões de alta concentração de arestas. Esse comportamento é importante porque se aproxima de propriedades observadas em vários grafos reais [7].

O fluxo do Graph500 é organizado em etapas. Primeiro, o benchmark gera uma lista de arestas, também chamada de grafo em tuplas. Nessa forma inicial, cada aresta é representada apenas por seus dois vértices. Essa representação é adequada para geração e distribuição inicial dos dados, mas não é a forma mais eficiente para executar uma travessia, pois a BFS precisa consultar rapidamente os vizinhos de cada vértice ativo.

A primeira etapa medida pelo benchmark é conhecida como Kernel 1. Ela consiste em construir uma estrutura de dados de grafo a partir da lista de arestas. Em implementações de BFS, essa estrutura frequentemente é uma variação de CSR (*Compressed Sparse Row*), na qual cada vértice possui um intervalo de posições que aponta para seus vizinhos. Em uma execução distribuída, essa construção também precisa decidir qual processo ou PE é responsável por cada vértice e por suas listas de adjacência. Portanto, o Kernel 1 mede a capacidade do sistema de reorganizar uma entrada irregular em uma estrutura adequada para travessia [7].

Depois da construção do grafo, o benchmark seleciona um conjunto de raízes para as execuções de BFS. A especificação evita escolher vértices isolados, pois uma BFS iniciada em um vértice sem arestas não exercitaria o sistema de forma representativa. Em geral, são escolhidas várias raízes, e a BFS é executada uma vez para cada uma delas. Isso reduz a chance de o resultado depender apenas de uma origem específica e permite calcular estatísticas como mínimo, máximo,

media e mediana de desempenho.

A segunda etapa principal é o Kernel 2, chamado de *Search*. Nessa etapa, o benchmark executa BFS a partir de cada raiz selecionada. O resultado esperado não é apenas a ordem de visita dos vértices, mas uma árvore de predecessores. Cada vértice alcançável deve registrar um pai válido, isto é, um vértice que o descobriu no nível anterior da busca. Vértices não alcançáveis a partir da raiz devem permanecer sem predecessor. Essa representação por predecessores é usada posteriormente para validar a correção da BFS [7].

A validação é uma parte essencial do Graph500. Uma implementação não pode apenas produzir um tempo de execução baixo; ela precisa demonstrar que a árvore de BFS gerada é correta. A validação verifica propriedades como: a raiz aponta para si mesma, vértices visitados possuem predecessores válidos, as arestas da árvore existem no grafo, e os níveis respeitam a propriedade da BFS. Essa etapa é especialmente importante em implementações paralelas, nas quais várias threads ou processos podem tentar descobrir o mesmo vértice ao mesmo tempo.

A principal métrica de desempenho reportada pelo Graph500 é TEPS (*Traversed Edges Per Second*). Essa métrica representa quantas arestas são atravessadas por segundo durante a BFS. Como a BFS executa pouca computação aritmética por aresta, TEPS é uma métrica mais apropriada do que operações de ponto flutuante por segundo. Ela reflete melhor a capacidade do sistema de percorrer estruturas irregulares, acessar memória, comunicar descobertas remotas e sincronizar a execução distribuída.

No contexto deste trabalho, o Graph500 é importante por dois motivos. Primeiro, ele fornece uma especificação clara para gerar grafos, executar BFS, validar o resultado e medir desempenho. Segundo, sua implementação de referência oferece uma base concreta para refatoração. A partir dela, é possível estudar como a BFS distribuída organiza vértices, fronteiras e predecessores, e então substituir ou reorganizar partes da comunicação usando NVSHMEM.

6 BFS em GPU

GPUs modernas são otimizadas para executar operações computacionalmente intensivas com muitos grupos de threads em paralelo e dependem de concorrência para esconder atrasos de memória. No modelo da NVIDIA, threads são agrupadas em *warps*, que executam uma mesma instrução sobre dados diferentes, e em blocos de threads, que podem compartilhar memória local no multiprocessador. Esse modelo favorece cargas de trabalho com grande quantidade de operações independentes e com acesso à memória bem organizado [10].

A BFS é atraente para GPUs porque a expansão de uma fronteira pode expor muito paralelismo. Em uma mesma iteração, vários vértices da fronteira atual podem ser expandidos simultaneamente, e as arestas incidentes a esses vértices também podem ser examinadas em paralelo. Quando a fronteira é grande, há trabalho suficiente para ocupar muitos multiprocessadores. Além disso, a BFS executa pouca computação aritmética por aresta; portanto, seu desempenho

depende principalmente de largura de banda, latencia de memoria, controle de concorrencia e eficiencia na organizacao das fronteiras [10].

Apesar disso, grafos esparsos tornam a BFS dificil em GPU. O padrao de acesso a memoria depende da estrutura do grafo, nao de um arranjo regular como em vetores ou matrizes densas. Ao expandir um vertice, as threads seguem listas de adjacencia que podem apontar para regioes distantes da memoria. Alem disso, grafos reais frequentemente possuem distribuicao de grau irregular, alguns vertices tem poucos vizinhos, enquanto outros concentram muitos. Se uma thread ou um warp recebe um vertice de grau muito maior que os demais, ocorre desbalanceamento de carga e parte da GPU pode ficar subutilizada [10].

Uma forma simples de paralelizar BFS e fazer cada iteracao inspecionar todos os vertices ou todas as arestas do grafo, procurando quais deles pertencem ao nivel atual. Essa estrategia se ajusta facilmente ao modelo de dados paralelos da GPU, mas pode executar trabalho quadratico no pior caso, pois o mesmo grafo e varrido repetidas vezes ao longo dos niveis da busca. Para grafos com diametro nao trivial, esse custo se torna alto. Uma implementacao eficiente deve examinar apenas os vertices e arestas que pertencem as fronteiras ativas de cada iteracao, mantendo custo linear em relacao ao numero de vertices e arestas, isto e, $O(|V| + |E|)$ [10].

No ponto de vista de desempenho, permitir muitas descobertas duplicadas de um mesmo vertice pode gerar trabalho redundante, pois o mesmo vertice pode ser inserido mais de uma vez na fronteira e sua lista de adjacencia pode ser expandida repetidamente. Por isso, implementacoes em GPU costumam usar mecanismos de filtragem, marcacao e remocao local de duplicatas para reduzir esse efeito [10].

Portanto, a BFS em GPU deve equilibrar tres objetivos. Primeiro, preservar a estrutura por niveis da BFS, garantindo que a proxima fronteira contenha apenas vertices descobertos a partir da fronteira atual. Segundo, manter eficiencia de trabalho, evitando varrer o grafo inteiro a cada iteracao. Terceiro, adaptar a execucao irregular do grafo ao modelo de hardware da GPU, organizando fronteiras, trabalho e comunicacao de forma a reduzir desbalanceamento, contencao e acessos inefficientes a memoria [10].

7 MPI

MPI (*Message Passing Interface*) e um padrao de comunicacao para programas paralelos em memoria distribuida. Em um programa MPI, a execucao e organizada em processos independentes, chamados normalmente de ranks. Cada rank possui seu proprio espaco de enderecamento e troca dados com os demais por meio de chamadas explicitas de comunicacao. Esse modelo e adequado para clusters, onde a memoria de uma maquina ou processo nao pode ser acessada diretamente por outro processo sem uma operacao de comunicacao definida [11].

O modelo de MPI se baseia em troca de mensagens. Em comunicacoes ponto a ponto, um rank envia dados e outro rank recebe esses dados. Operacoes como `MPI_Send` e `MPI_Recv` representam esse padrao basico. Tambem existem

versoes nao bloqueantes, como `MPI_Isend` e `MPI_Irecv`, que permitem iniciar uma transferencia e continuar a execucao enquanto a comunicacao progride. Esse tipo de comunicacao e importante em aplicacoes paralelas porque pode permitir sobreposicao entre computacao e transferencia de dados.

MPI tambem define operacoes coletivas, nas quais todos os ranks de um comunicador participam. Exemplos comuns incluem `MPI_Barrier`, usada para sincronizacao, `MPI_Bcast`, usada para difundir dados de um rank para os demais, e `MPI_Reduce` ou `MPI_Allreduce`, usadas para combinar valores produzidos localmente por cada rank. Em algoritmos iterativos, essas operacoes aparecem frequentemente para calcular condicoes globais de parada, agregar estatisticas ou garantir que todos os processos avancem para a proxima fase de execucao.

Em BFS distribuida, MPI pode ser usado para coordenar a divisao do grafo entre ranks e para comunicar descobertas de vertices remotos. Quando um rank expande um vertice local e encontra uma aresta para um vertice pertencente a outro rank, essa descoberta precisa ser enviada ao dono do vertice. Uma implementacao baseada apenas em MPI normalmente acumula essas descobertas em buffers, troca mensagens entre ranks ao final de uma fase e, depois, cada rank atualiza sua propria fronteira local. Esse modelo preserva a semantica da BFS por niveis, mas pode introduzir custos de empacotamento, sincronizacao e troca de mensagens.

No Graph500, MPI aparece naturalmente porque o benchmark foi projetado para execucoes em memoria distribuida. Ele pode ser usado para inicializar a execucao paralela, distribuir o grafo, realizar comunicacoes durante a construcao da estrutura de dados, selecionar raizes, executar reducoes para metricas e validar resultados. Na implementacao original usada como referencia neste trabalho, MPI tambem aparece na construcao do CSR distribuido e no controle global do fluxo do benchmark.

No contexto deste trabalho, MPI e importante mesmo quando a travessia usa NVSHMEM. As duas tecnologias atuam em niveis diferentes. MPI fornece o modelo de processos, a inicializacao distribuida e varias operacoes coletivas no host. NVSHMEM, por outro lado, permite que kernels CUDA realizem operacoes remotas diretamente entre PEs. Assim, uma implementacao pode usar MPI para coordenar a execucao em alto nivel e NVSHMEM para reduzir a dependencia do host durante as atualizacoes finas da BFS.

8 NVSHMEM

NVSHMEM e uma biblioteca de comunicacao para GPUs NVIDIA baseada no modelo OpenSHMEM. O objetivo eh oferecer um espaco de memoria global entre GPUs, onde cada elemento de processamento possui uma memoria local, que pode ser enderecado globalmente atraves de operacoes de comunicacao. O principal diferencial do NVSHMEM e o motivo de ser utilizado nesse trabalho eh que as operacoes podem ser emitidas diretamente de um kernel CUDA, sem envolver a CPU [8].

As unidades de processamento sao divididasaaa em PEs (Process Unit). A

biblioteca define regioes de memoria simetricas, essas regioes sao alocadas coletivamente por todas as PEs. A simetria permite que uma PE, calcule o endereco logico global correspondente a um endereco logico local, e um indentificador de PE. O NVSHMEM fornece operacoes remotas como leituras e escritas, e atomicos.

Pelas suas caracteristicas a biblioteca NVSHMEM e relevante para algoritmos irregulares. Em aplicacoes densas ou estruturadas, a comunicacao pode ser prevista e agrupada em mensagens grandes. Em grafos, por outro lado, os destinos das atualizacoes sao descobertos durante a execucao. Ao expandir uma fronteira de BFS, uma thread pode visitar uma aresta, identificar que o vertice vizinho pertence a outro PE e precisar atualizar o estado desse vertice remoto. Com NVSHMEM, essa atualizacao pode ser expressa diretamente no kernel, usando uma operacao remota ou atomica, sem retornar primeiro ao host para empacotar mensagens [8].

9 Refatoracao do kernel Graph500

Um dos objetivos deste trabalho era reimplementar a versao de referencia do benchmark Graph500 utilizando NVSHMEM. A implementacao segue um modelo hibrido: o host controla a sincronizacao entre fronteiras da BFS, enquanto um kernel CUDA executa a expansao da fronteira atual e utiliza operacoes NVSHMEM para atualizar dados remotos diretamente entre PEs. Assim, a CPU ainda participa da coordenacao global da busca, mas deixa de ser necessaria para cada descoberta remota de vertice.

10 Estrutura do Graph500

Na versao de referencia do Graph500, o fluxo principal do benchmark e separado da implementacao da travessia por uma pequena interface. O programa principal gera o grafo, escolhe as raizes da BFS, chama uma rotina para construir a estrutura interna do grafo e, para cada raiz, chama a funcao de BFS.

A primeira etapa e a conversao do grafo gerado em tuplas para uma representacao CSR distribuida. O Graph500 distribui os vertices de forma ciclica entre os ranks. O dono de um vertice global v e calculado por $v \bmod P$, onde P e o numero de PEs, e o indice local e calculado por v / P .

Durante o algoritmo da BFS enquanto a fronteira global nao estiver vazia, o host lanca o kernel de expansao da fronteira. Dentro desse kernel, cada thread processa vertices da fronteira local. Para cada vertice, a thread percorre sua lista de adjacencia no CSR, calcula o PE dono do vizinho e tenta marcar esse vizinho como descoberto.

Quando uma descoberta e bem-sucedida, o kernel tambem insere o vertice na proxima fronteira do PE que possui esse vertice. Para isso, usa uma operacao atomica remota de incremento sobre o contador da proxima fronteira e, em seguida, escreve o indice local do vertice descoberto na posicao obtida. Esse

padrao e o ponto central do uso de NVSHMEM na BFS, a thread que encontra uma aresta remota nao precisa retornar ao host nem empacotar uma mensagem MPI. Ela atualiza diretamente o estado remoto necessario para que o PE dono processe esse vertice no proximo nivel.

Ao final da expansao, o kernel garante a conclusao das operacoes remotas, e o host volta a sincronizar os PEs. O contador da proxima fronteira e copiado da GPU para a CPU, os ranks executam uma reducao global para somar os tamanhos das fronteiras locais, e a BFS continua apenas se essa soma for maior que zero.

Essa divisao entre host e device torna a implementacao mais simples de validar, pois preserva a estrutura classica da BFS por fronteiras. Ao mesmo tempo, ela mostra uma limitacao importante, embora as descobertas remotas ocorram no device, a cada nivel ainda existem sincronizacoes com a CPU, copias de contadores e reducoes globais.

11 BFS somente kernel

No trabalho tambem foi feita uma implementacao que utiliza somente o kernel, sem sincronizacao feita pelo host entre os niveis da BFS. O codigo segue a mesma estrutura conceitual da versao com host: existem duas fronteiras, um vetor de predecessores, contadores de fronteira e operacoes NVSHMEM para atualizar vertices remotos. A diferenca principal e que o laco da BFS deixa de estar no host e passa a estar dentro de um unico kernel CUDA.

A principal vantagem dessa abordagem e reduzir o custo fixo por nivel da BFS. Em grafos de pequeno diametro, esse custo pode ser menos dominante, mas em execucoes com muitos niveis a sincronizacao host-device repetida pode afetar o tempo total. Ao manter o laco dentro do kernel, a implementacao evita multiplos lancamentos de kernel, copias de contadores para a CPU e reducoes MPI entre niveis. Isso aproxima a BFS do modelo de comunicacao proposto por NVSHMEM, no qual a GPU descobre o destino da comunicacao e tambem emite a operacao remota.

Por outro lado, essa versao tambem introduz restricoes. Como a sincronizacao entre blocos e feita dentro de um kernel cooperativo, a quantidade de blocos deve ser escolhida de forma que todos possam progredir sem causar deadlock. A implementacao precisa considerar a ocupacao do dispositivo e o numero de PEs que compartilham a mesma GPU. Alem disso, o uso de barreiras globais no device ainda mantem a BFS rigidamente separada por niveis; a versao somente kernel remove a CPU do caminho entre niveis, mas nao remove a necessidade algoritmica de sincronizar a fronteira global.

Assim, a versao somente kernel representa um passo alem da implementacao host/device. Ela preserva a mesma semantica de descoberta por fronteiras e o mesmo uso de atomicos remotos para definir predecessores, mas desloca a coordenacao da BFS para dentro do kernel. Essa mudanca torna a implementacao mais alinhada ao objetivo de usar NVSHMEM para comunicacao iniciada pela GPU e serve como base para avaliar quanto do custo da versao anterior vinha

da participacao recorrente do host.

References

- [1] Leman Akoglu, Hanghang Tong, and Danai Koutra. Graph based anomaly detection and description: A survey. *Data Mining and Knowledge Discovery*, 29(3):626–688, 2015.
- [2] Albert-Laszlo Barabasi and Zoltan N. Oltvai. Network biology: Understanding the cell’s functional organization. *Nature Reviews Genetics*, 5(2):101–113, 2004.
- [3] Scott Beamer, Krste Asanovic, and David Patterson. Direction-optimizing breadth-first search. In *Proceedings of the International Conference on High Performance Computing, Networking, Storage and Analysis*. IEEE Computer Society Press, 2012.
- [4] Sergey Brin and Lawrence Page. The anatomy of a large-scale hypertextual web search engine. In *Proceedings of the Seventh International Conference on World Wide Web*, pages 107–117. Elsevier Science Publishers B. V., 1998.
- [5] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. *Introduction to Algorithms*. MIT Press, 3 edition, 2009.
- [6] David Easley and Jon Kleinberg. *Networks, Crowds, and Markets: Reasoning About a Highly Connected World*. Cambridge University Press, 2010.
- [7] Graph 500 Steering Committee. Graph 500 Benchmark Specification: Graph 500 Benchmarks 1 (Search) and 2 (Shortest Path), 2017. Acesso em: 25 jun. 2026.
- [8] Chung-Hsing Hsu, Neena Imam, Akhil Langer, Sreeram Potluri, and Chris J. Newburn. An initial assessment of NVSHMEM for high performance computing. In *2020 IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW)*, pages 1–10. IEEE, 2020.
- [9] Grzegorz Malewicz, Matthew H. Austern, Aart J. C. Bik, James C. Dehnert, Ilan Horn, Naty Leiser, and Grzegorz Czajkowski. Pregel: A system for large-scale graph processing. In *Proceedings of the 2010 ACM SIGMOD International Conference on Management of Data*, pages 135–146. ACM, 2010.
- [10] Duane Merrill, Michael Garland, and Andrew Grimshaw. Scalable gpu graph traversal. In *Proceedings of the 17th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming*, pages 117–128. ACM, 2012.

- [11] Message Passing Interface Forum. MPI: A message-passing interface standard version 4.1, 2023. Acesso em: 1 jul. 2026.
- [12] Richard C. Murphy, Kyle B. Wheeler, Brian W. Barrett, and James A. Ang. Introducing the graph 500. In *Cray User's Group*. Sandia National Laboratories, may 2010. SAND2010-3263C. Acesso em: 25 jun. 2026.